

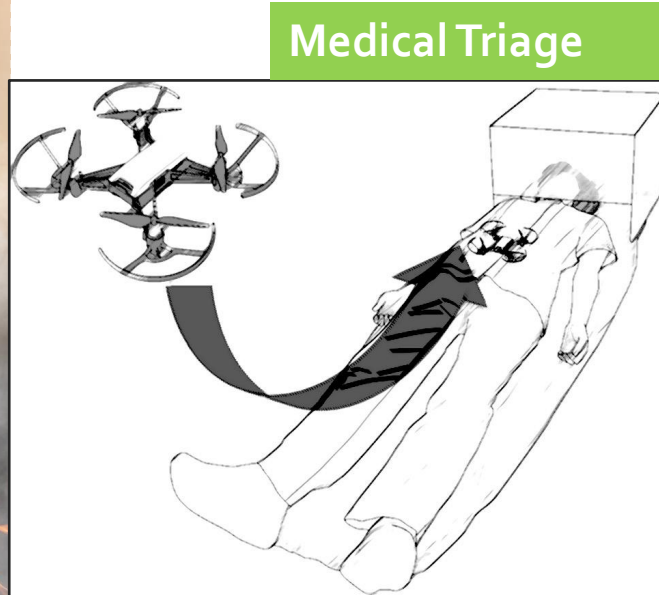
EdgeRL: Reinforcement learning-driven deep learning model inference optimization at edge

Motahare Mounesan, Xiaojie Zhang, Saptarshi Debroy

Visual Computing Applications For Mission-critical Use Cases



Disaster Response



Medical Triage

Requires just-in-time edge environment

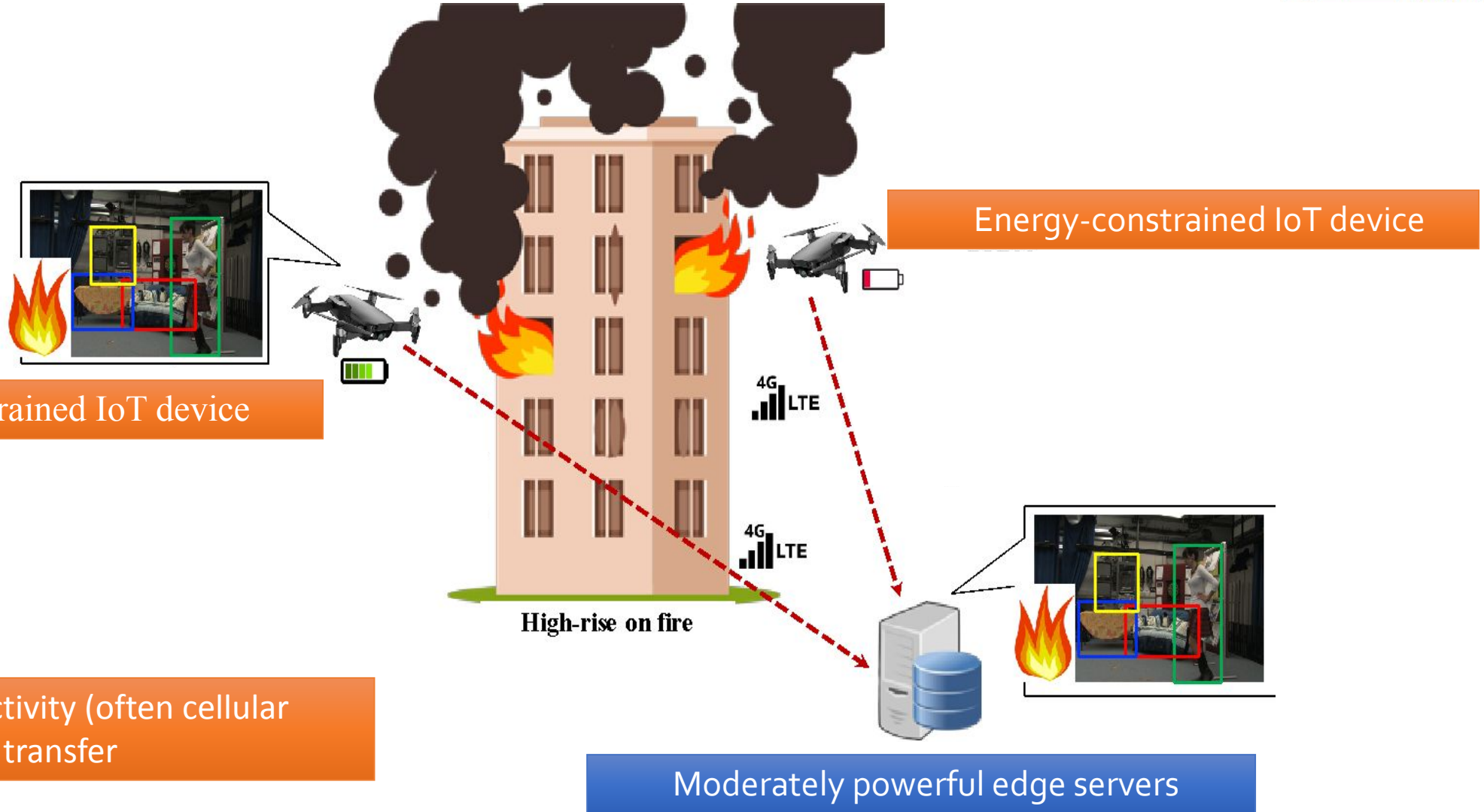
Strict latency requires **real-time** DNN inference



Tactical Scenarios



Components – Fire Rescue Use Case



Computation options

Local only:

- Longer latency ☐ **violation of real-time latency requirement**
- ☐ **too much energy consumption**

Remote only:

Sending large/high quality image over low data rate uplink connections

- ☐ high transmission time
- ☐ **violation of real-time latency requirement**

DJI Inspire 2



Full HD (e.g., Video surveillance, video gaming)

Technology	Upload Range (in Mbps)
4G LTE	1.8 – 12.0
5G	3.6 – 18.9



Edge Node

Max flight time < 70 min

Computation options

Local only:

Longer latency ☐ **violation of real-time latency requirement**
☐ **too much energy consumption**

Remote only:

Sending large/high quality image over low data rate uplink connections

☐ high transmission time
☐ **violation of real-time latency requirement**

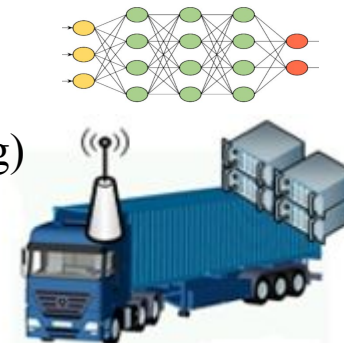
DJI Inspire 2



Full HD (e.g., Video surveillance, video gaming)

Technology	Upload Range (in Mbps)
4G LTE	1.8 – 12.0
5G	3.6 – 18.9

Max flight time < 30 min



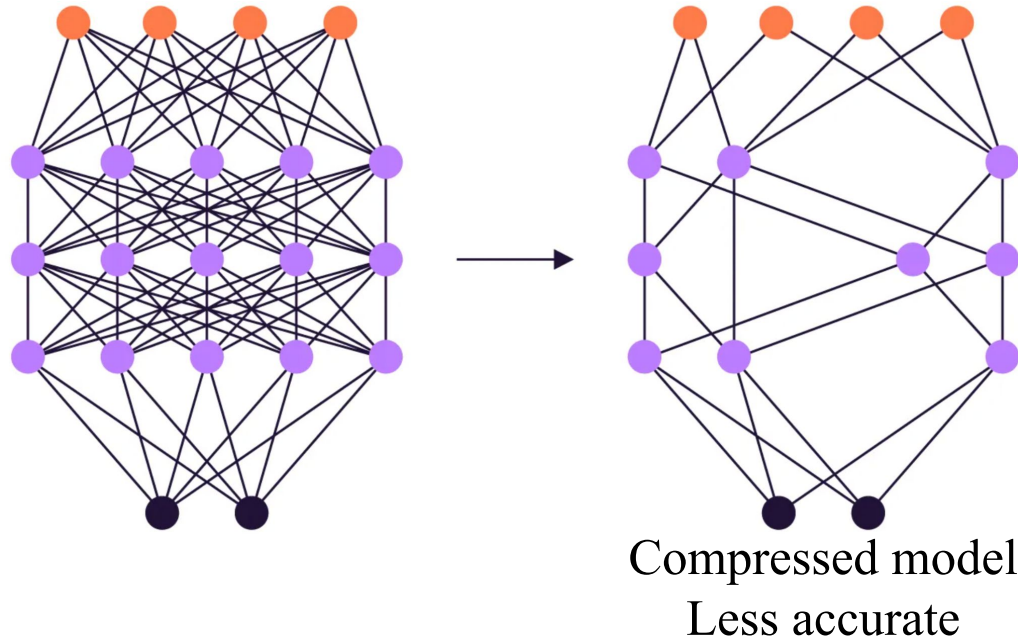
Edge Node

1. *Model Architecture optimization*
2. *Runtime Optimization*

Model Architecture Optimization

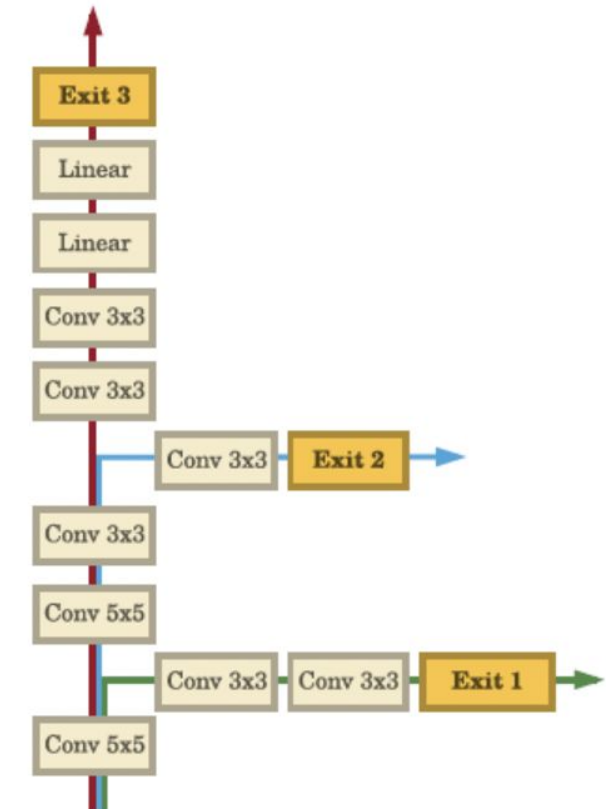
Reducing the size of deep neural network models while preserving performance, typically through techniques like *pruning*, *quantization*, and *knowledge distillation*:

DNN Compression



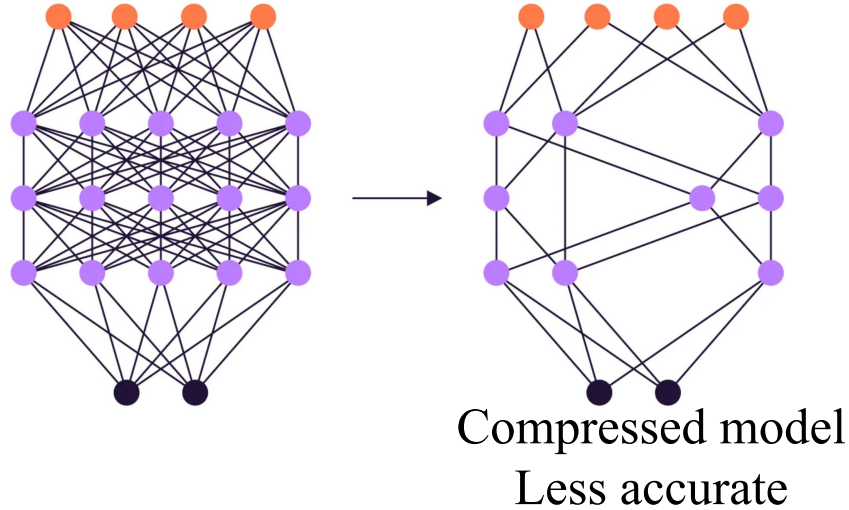
Accelerating inference and cutting computational costs by enabling models to make predictions before completing all layers based on specific criteria:

Early Exit

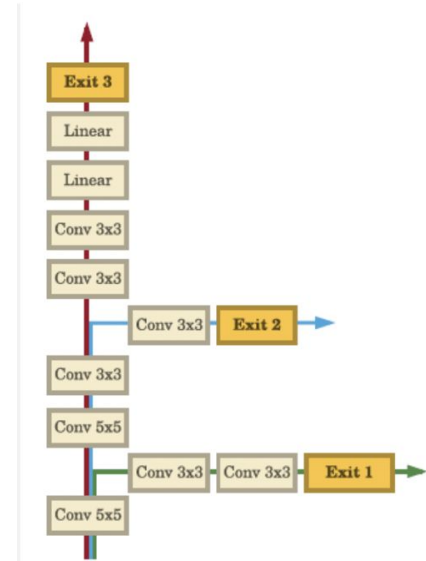


Model Architecture Optimization

Reducing the size of deep neural network models while preserving performance, typically through techniques like *pruning*, *quantization*, and *knowledge distillation*:
DNN Compression



Accelerating inference and cutting computational costs by enabling models to make predictions before completing all layers based on specific criteria:
Early Exit

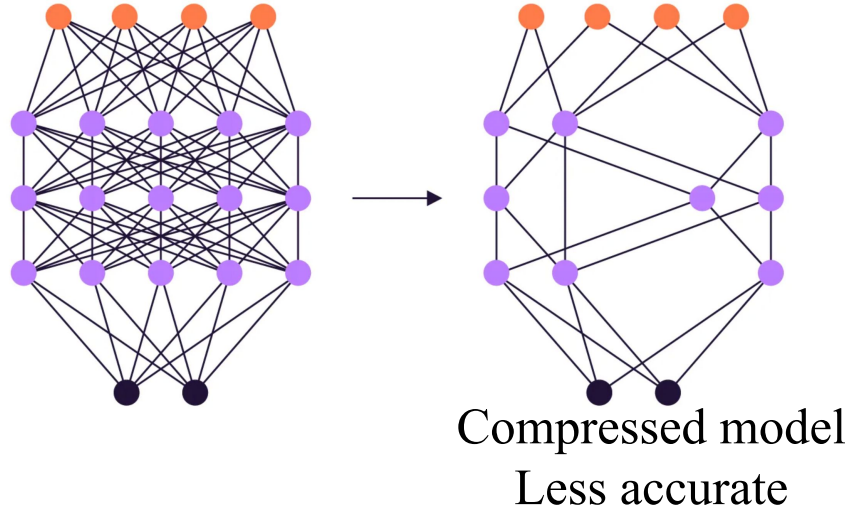


Different *Versions*
of the model

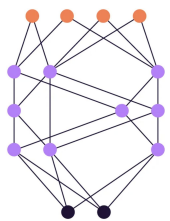
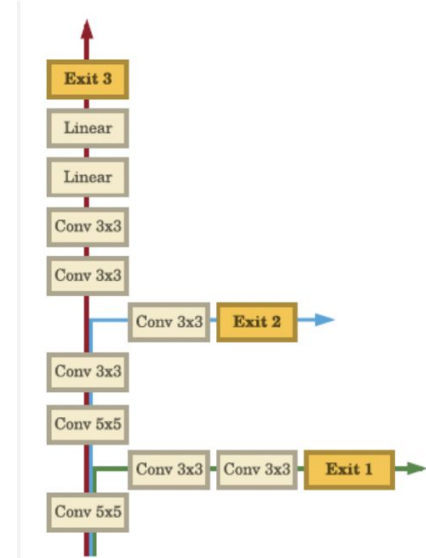
different in Accuracy, Latency and
consequently Energy Consumption

Model Architecture Optimization

Reducing the size of deep neural network models while preserving performance, typically through techniques like *pruning*, *quantization*, and *knowledge distillation*:
DNN Compression



Accelerating inference and cutting computational costs by enabling models to make predictions before completing all layers based on specific criteria:
Early Exit

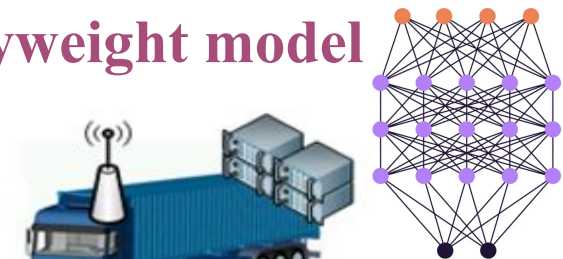


Lightweight model



DJI Inspire 2

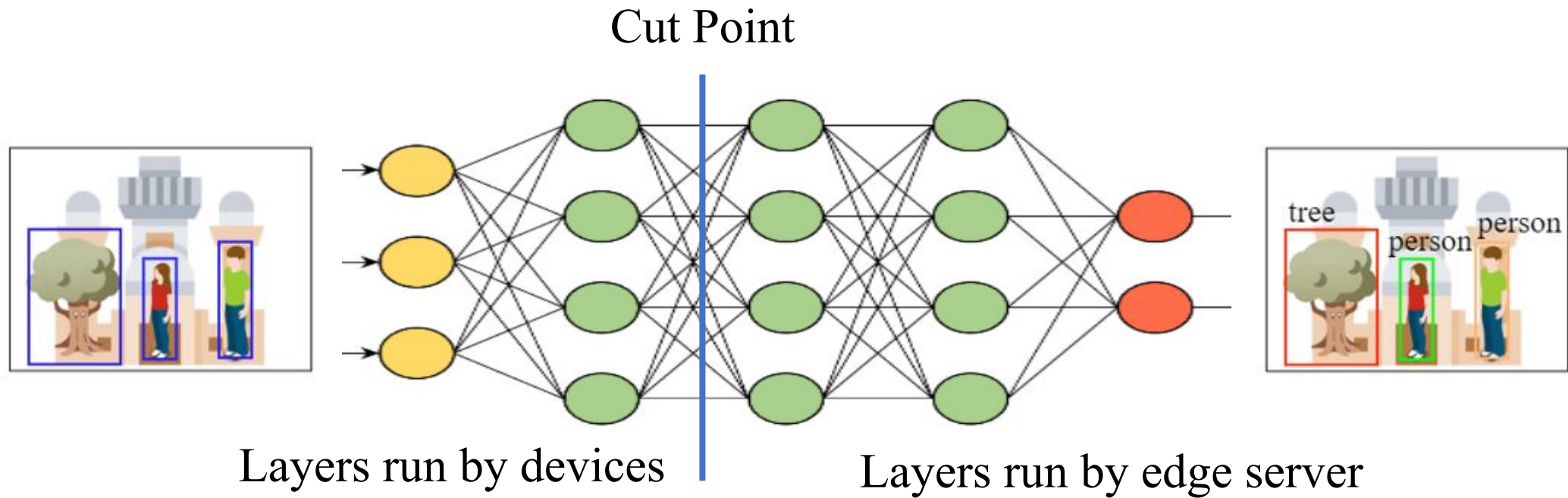
Heavyweight model



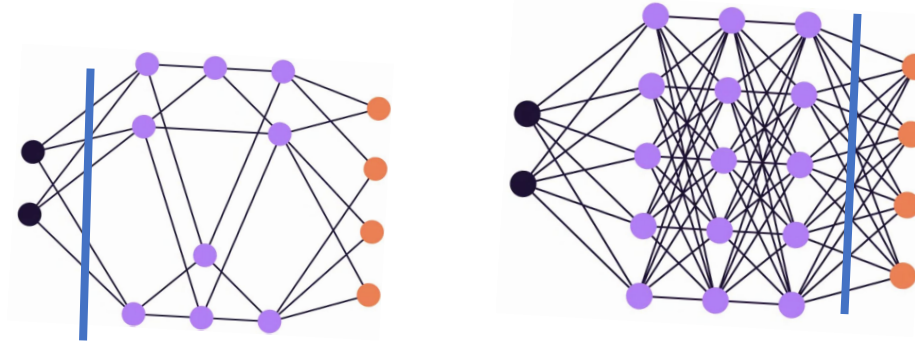
Edge Node

Runtime Optimization

Partial task offloading between IoT devices and edge servers: *DNN partitioning*



Novel approach



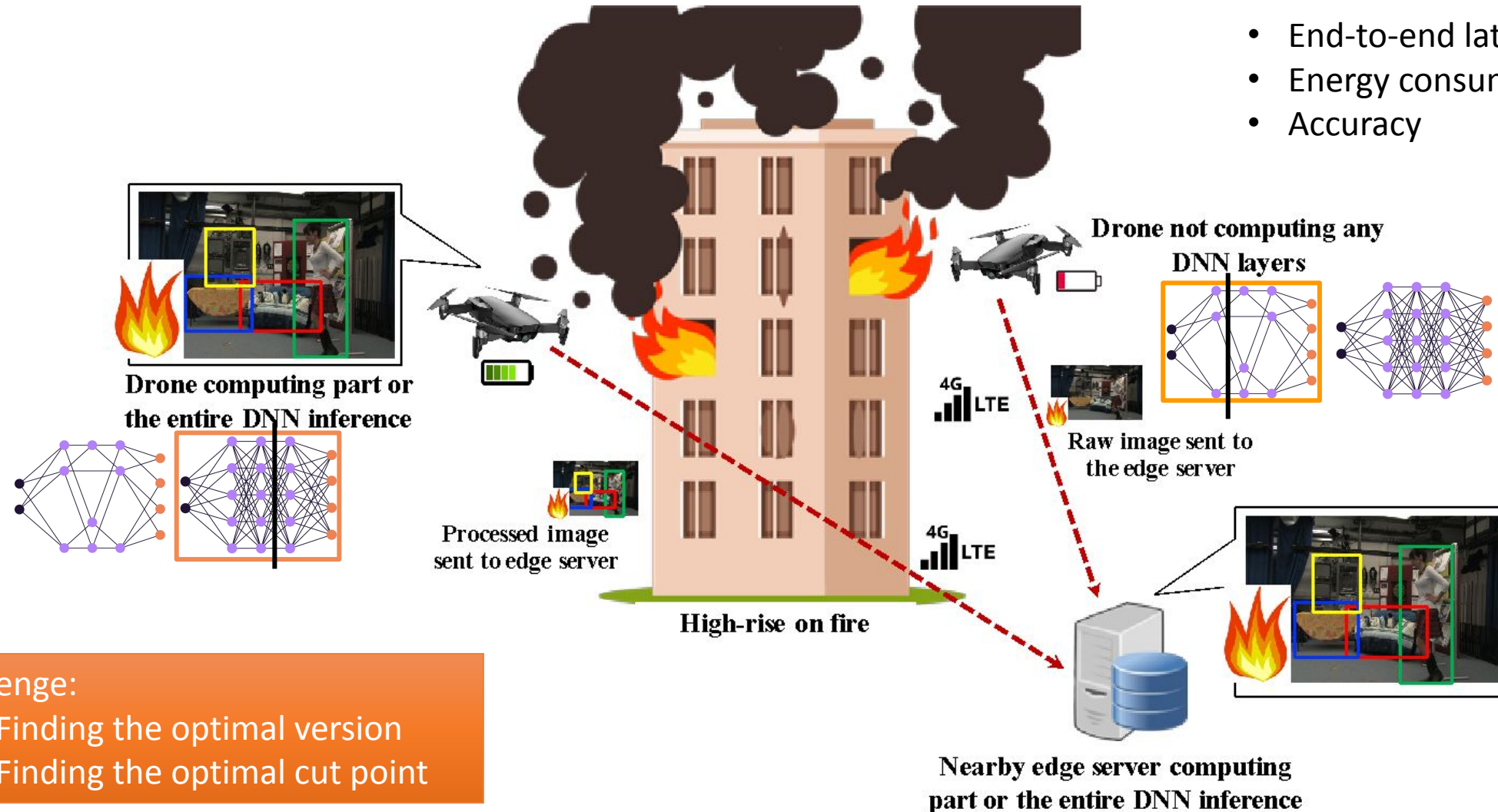
Combining these two ideas gives us two level of freedom:

1. Using the results of model architecture optimizations as various **versions**, deciding on the optimal version to run
2. Deploying collaborative DNN inference on the chosen **version**
[Deciding on the **cut point**]

Partial Task Offloading - Fire Rescue Use Case

DNN for object detection

- End-to-end latency
- Energy consumption
- Accuracy



Challenge:

1. Finding the optimal version
2. Finding the optimal cut point

Problem Evidence Analysis

Testbed Hardware device-server

Edge Server:

**Dell PowerEdge desktop
with 16 cores and 3.2
GHz CPU frequency**



IoT devices:

NVIDIA Jetson TX2
[most power-efficient AI
computing device among its
similar models]

Benchmarking with popular DNNs:

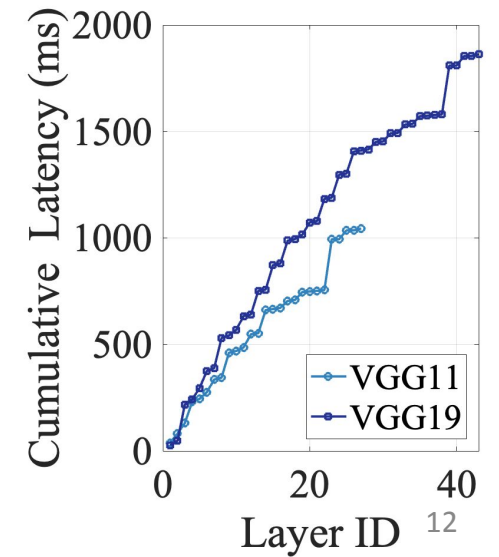
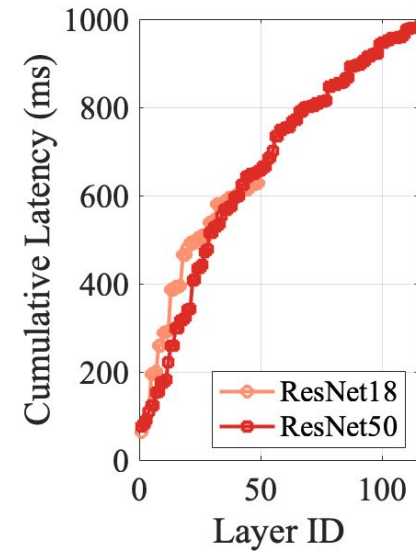
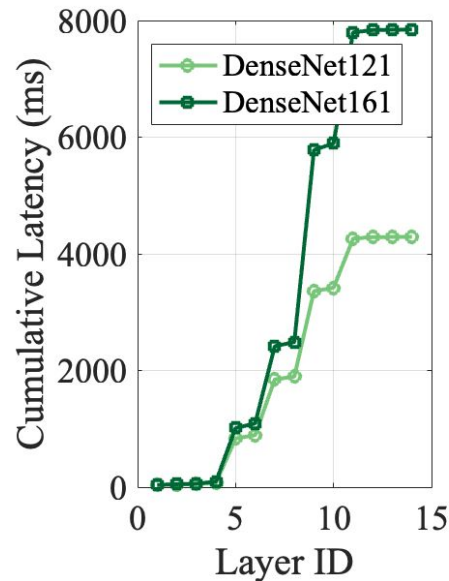
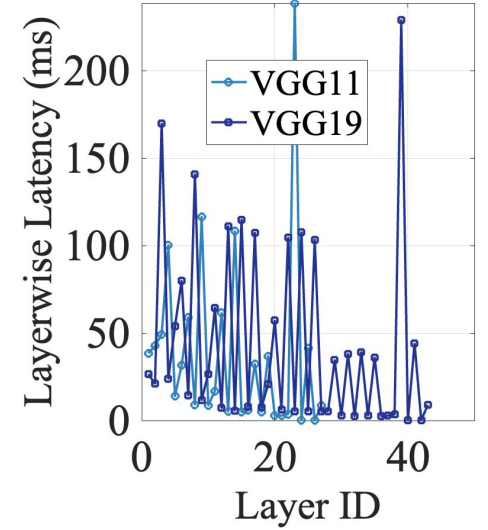
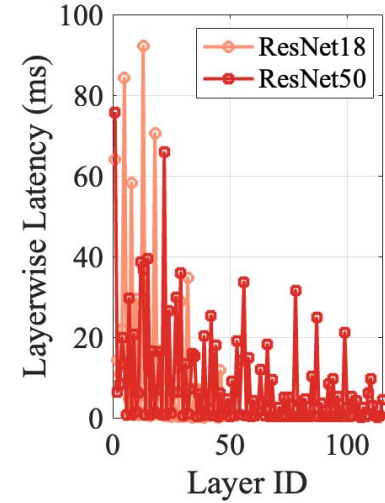
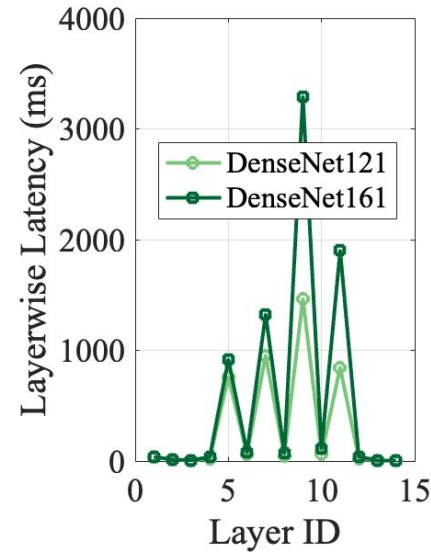
- VGG
- ResNet
- DenseNet

**Higher Level Partitioning is
considered
[No cut inside a Dense Block]**

Model	Accuracy (acc1)	Latency (ms)	Energy (J)
vgg11	69.04%	1044.48	6.17
vgg19	72.40%	1862.89	11.83
densenet121	74.43%	4292.17	28.00
densenet161	77.11%	7845.49	50.99
resnet18	69.76%	627.59	3.73
resnet50	76.15%	984.62	7.46

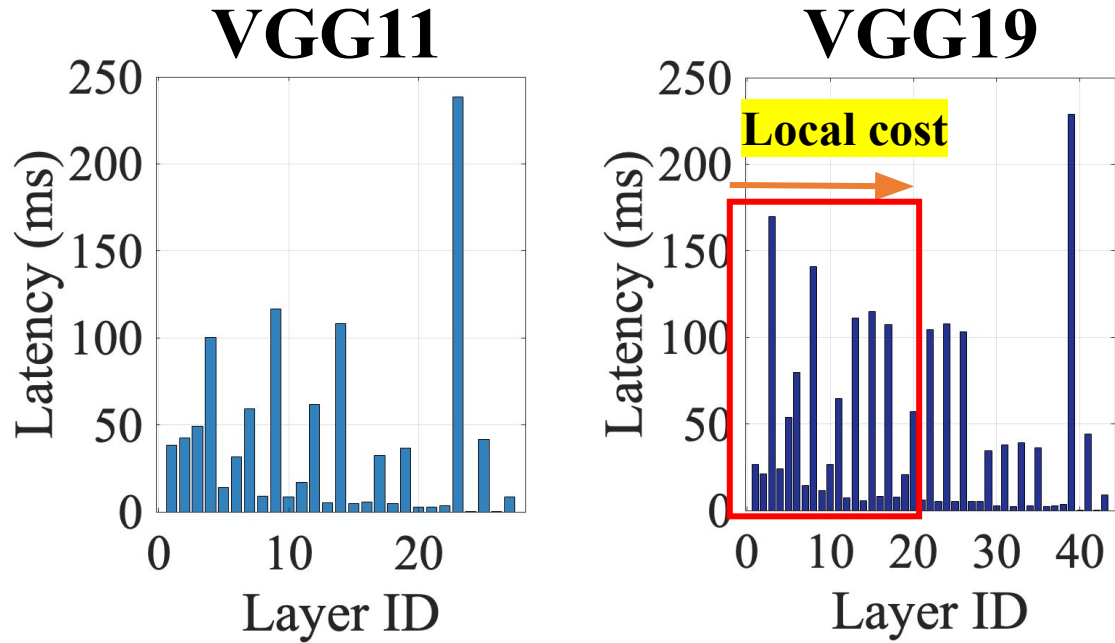
Latency comparison of versions

Differences stem from **complexity of layers** or the **number of layers** or **both**

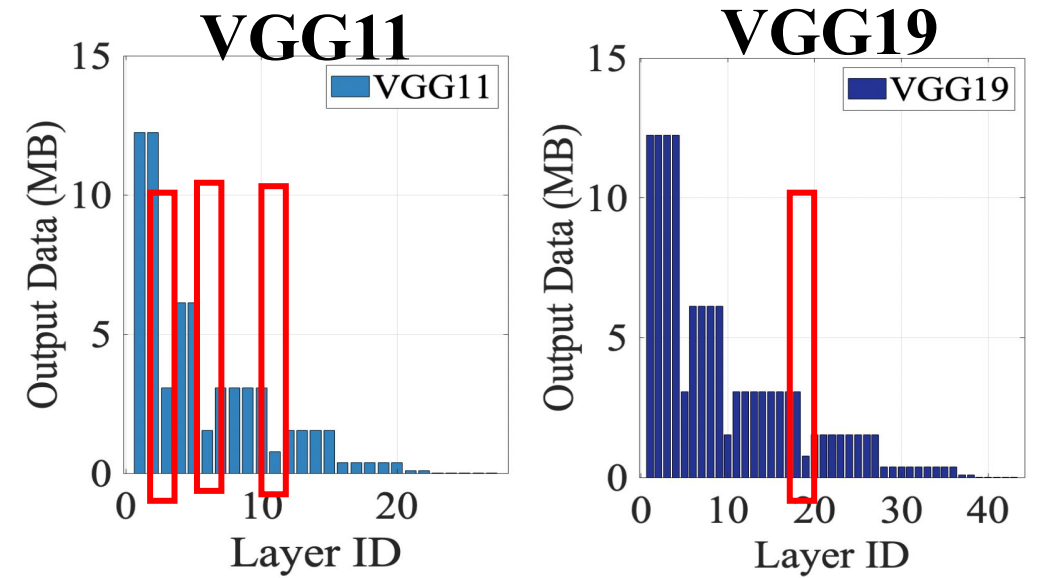


VGG layer-wise latency vs output data size

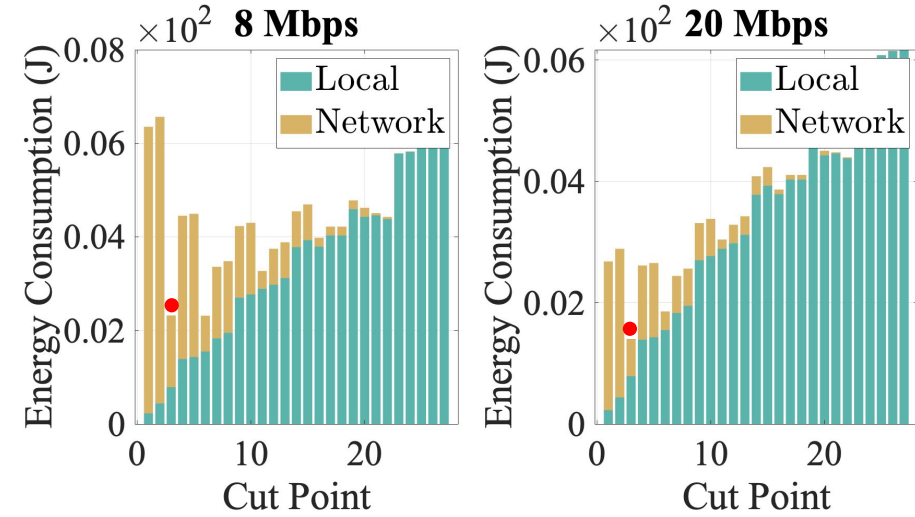
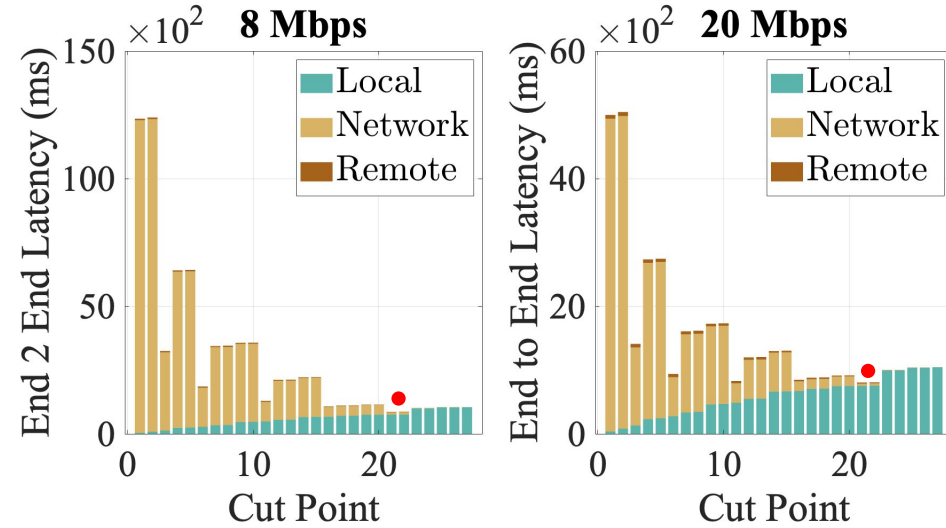
Layerwise Latency



Output Data Size

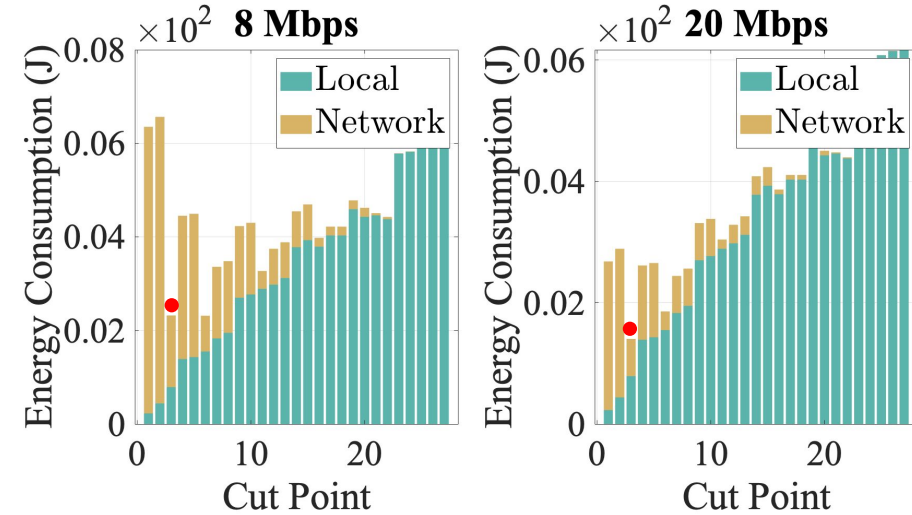
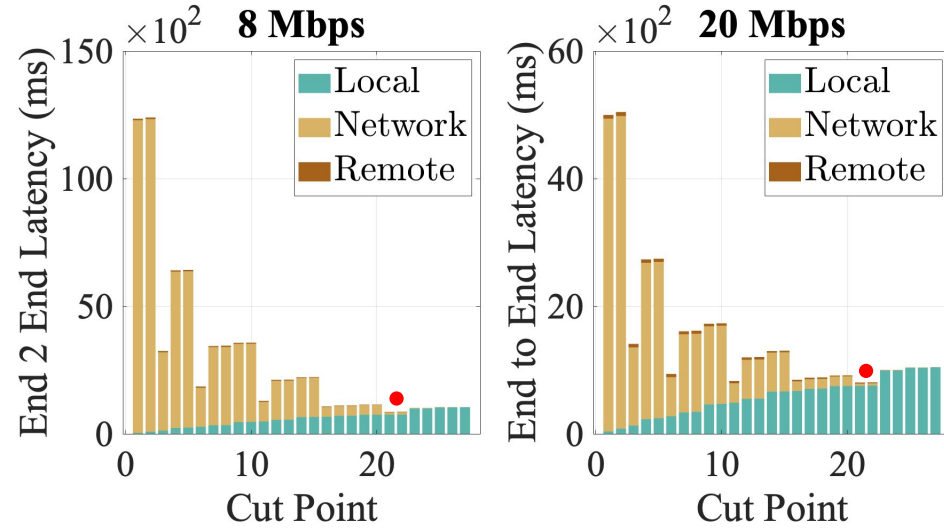


VGG - Latency VS Energy trade off

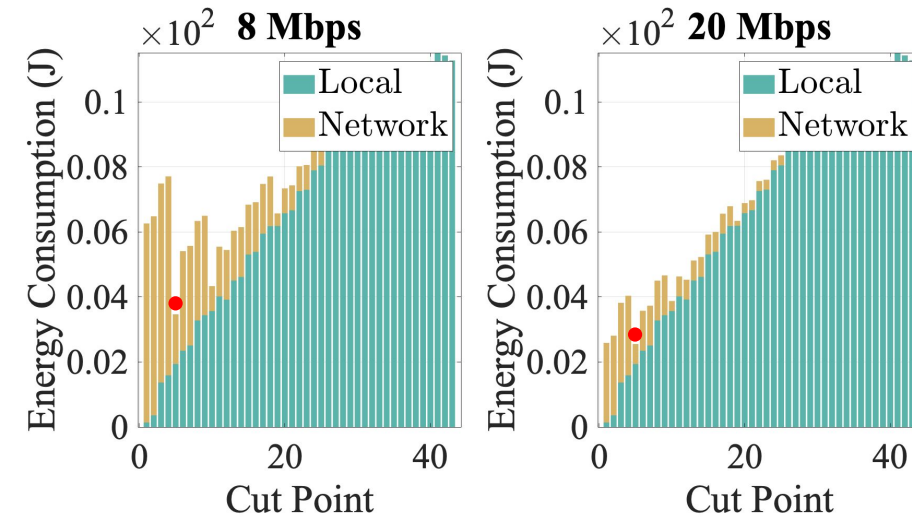
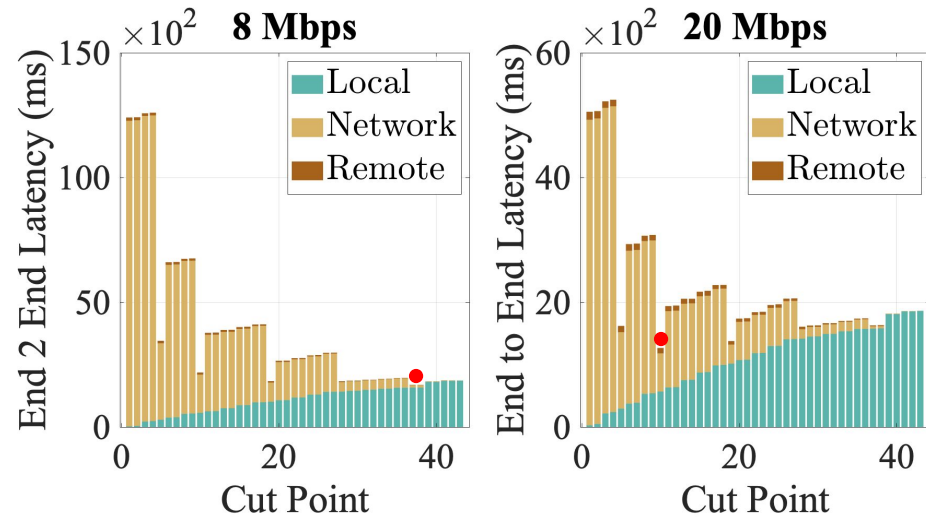


VGG11

VGG - Latency VS Energy trade off



VGG11



VGG19

Problem Evidence Analysis

For energy-constrained IoT devices running latency-sensitive applications, there is often a trade-off between end-to-end latency and energy consumption and accuracy.

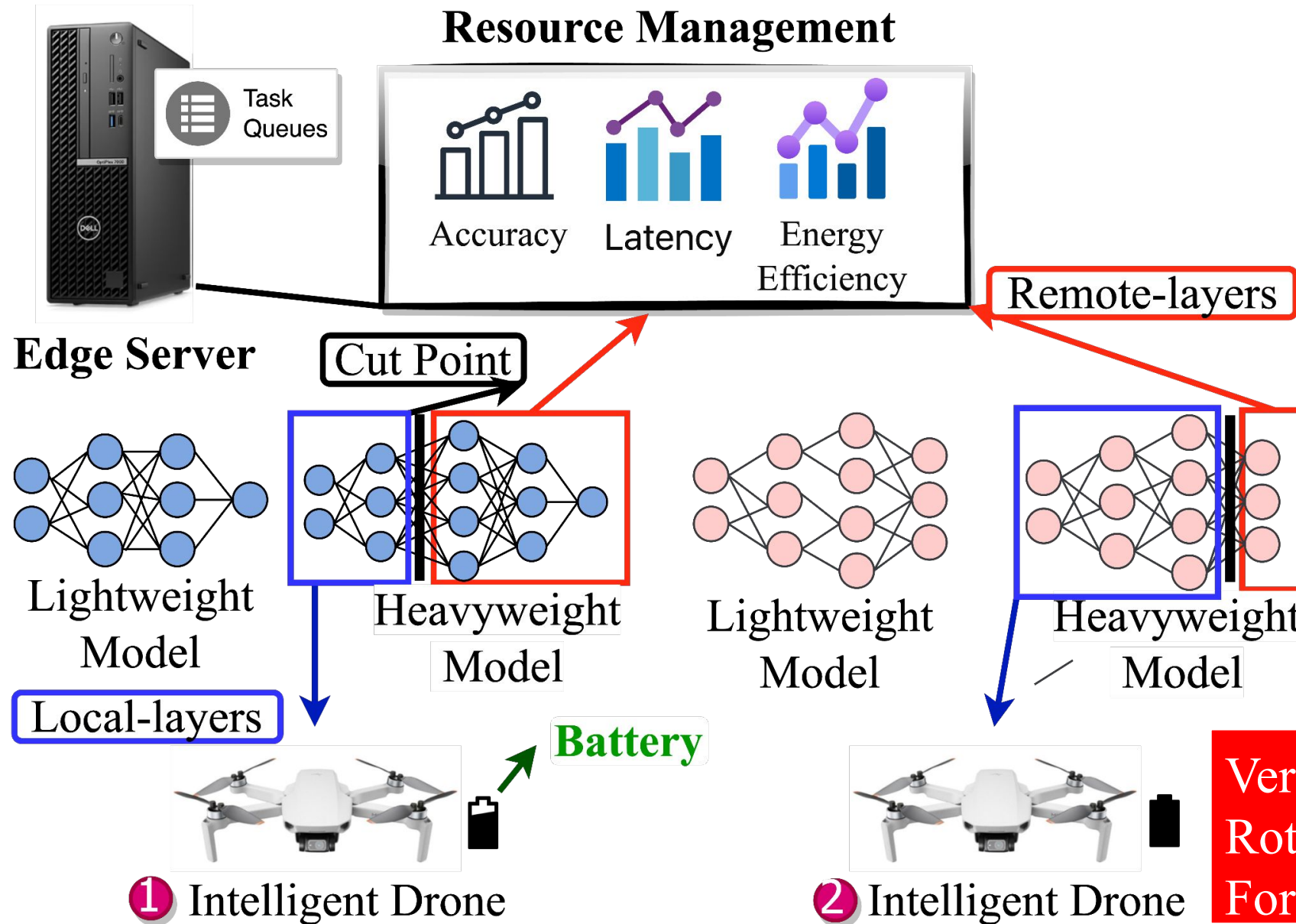
*Version and Cut point selection greatly impacts the end-to-end latency and energy consumption but should also be cognizant of **availability of resources***



Infer-EDGE

*Optimized DNN Inference in
'Just-in-time' Edge Environments*

Infer-EDGE System Architecture for Collaborative Inference



Decisions to be made

**Version
Cut-Point**

DNN Model

IoT devices chose the optimal version/cut point based on their requirements

Optimal cut point : Ideal trade-off between the **local costs** and **transmission costs**

**Vertical
Rotational
Forward**

Problem Formulation

Energy = local computation energy + Transmission energy

Latency = local latency + Transmission latency + Server-side latency

Accuracy : benchmarked over sample data

Long term optimization problem that jointly **selects the version and cut point** aims to:

- Minimize the *long-term average device energy consumption*
- Satisfy:
 - *long-term average end-to-end latency requirement*
 - *long-term average accuracy requirement*

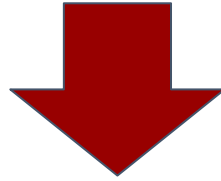
of DNN inference

$$\begin{aligned}
 & \min_{(M_{i,j}^l \text{ for each } U_k)} \quad \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \sum_{U_k \in \mathcal{U}} E_{i,j}^l(U_k) \\
 & \text{s.t.} \quad j \in V_i, l \in L_{i,j}, \\
 & \quad M_{i,j}^{acc} \geq \tau_i^{acc}, \\
 & \quad T_{i,j}^l(U_k) \leq \tau_i^{latency}.
 \end{aligned}$$

Solution Strategy

Analytical, numerical approaches and heuristics :

- Assumptions about system characteristics: *job arrival distributions, server queue length distributions, device energy consumption model, and end-to-end latency models*



Reinforcement learning

learn through system dynamic by trial and error

Advantage Actor Critic Method: efficient, sample efficient, stable

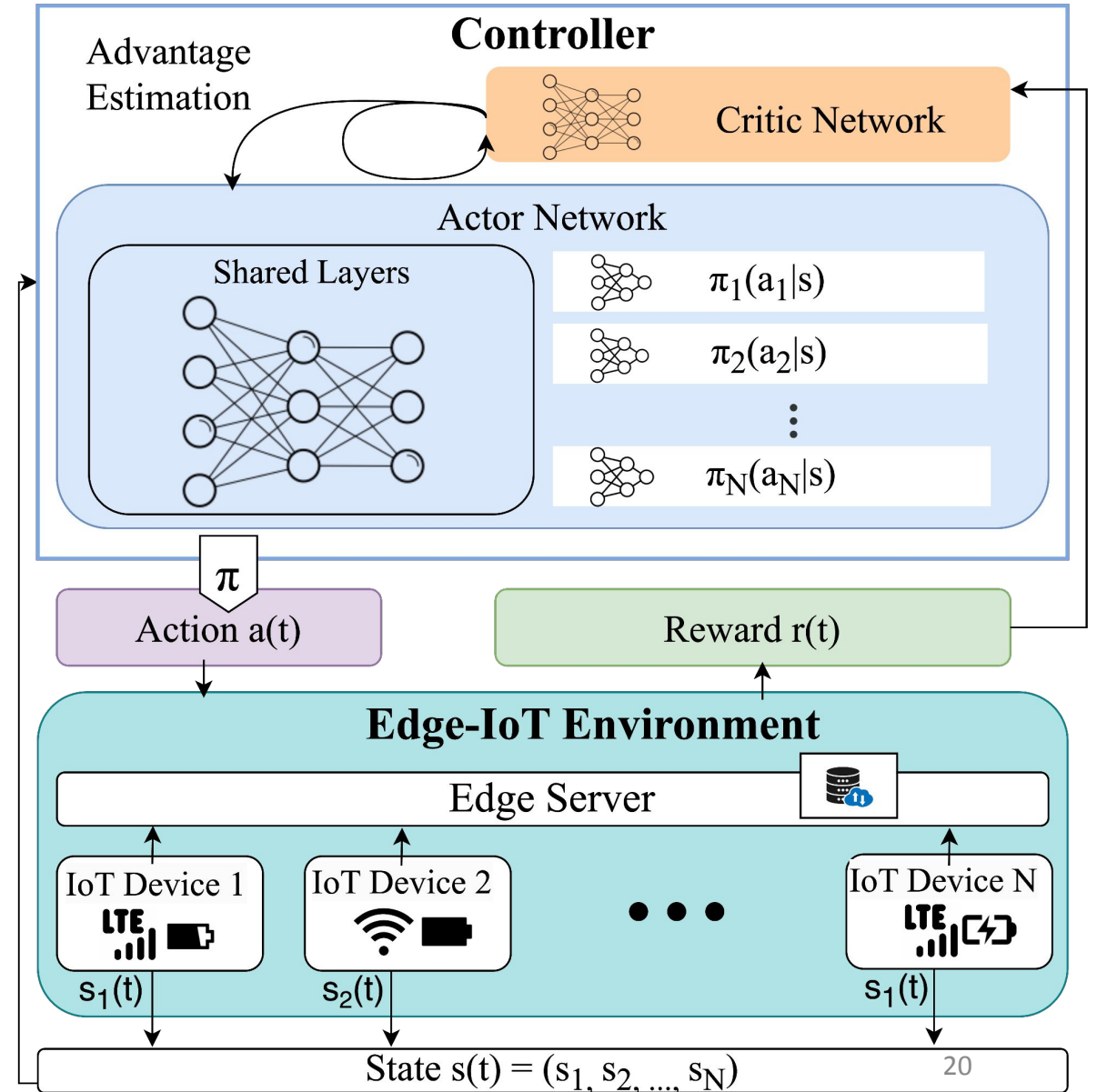
Solution Strategy

State:

Information on *battery level of the devices, available transmission rate, DNN model, Drone activity profile* for next time slot

Action:

Execution profile (version, cut point) selection for all devices [dependent]



Solution Strategy

Reward:

Weighted average of accuracy, latency, energy scores.

$$R(t) = \frac{1}{|\mathcal{U}|} \sum_{k \in \mathcal{U}} w_1 \cdot \mathcal{A}(M_{i,j}) + w_2 \cdot \mathcal{L}(M_{i,j}^l, U_k) + w_3 \cdot \mathcal{E}(M_{i,j}^l, U_k)$$

$$\mathcal{A}(M_{i,j}) = \frac{1}{1 + e^{-a \cdot (M_{i,j}^{acc} - b)}}$$

$$\mathcal{E}(M_{i,j}^l, U_k) = 1 - \frac{E_{i,j}^l(U_k)}{E_{i,j}^{Li}(U_k)}$$

$$\mathcal{L}(M_{i,j}^l, U_k) = 1 - \frac{T_{i,j}^l(U_k)}{T_{i,j}^{Li}(U_k)}$$

*Compare to
local-only
execution plan*

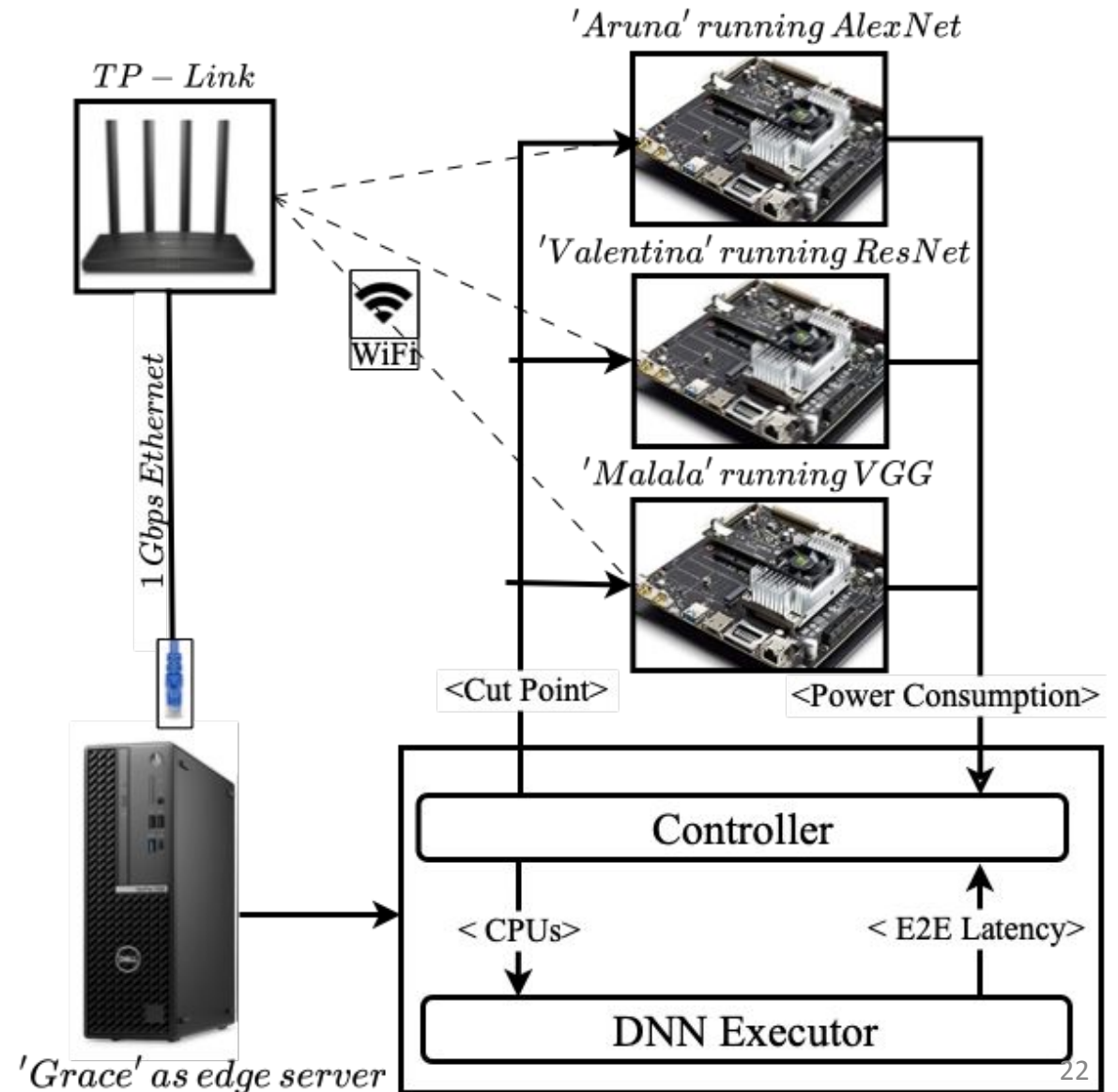
$$\sum_{i=1}^3 w_i = 1$$

Evaluation

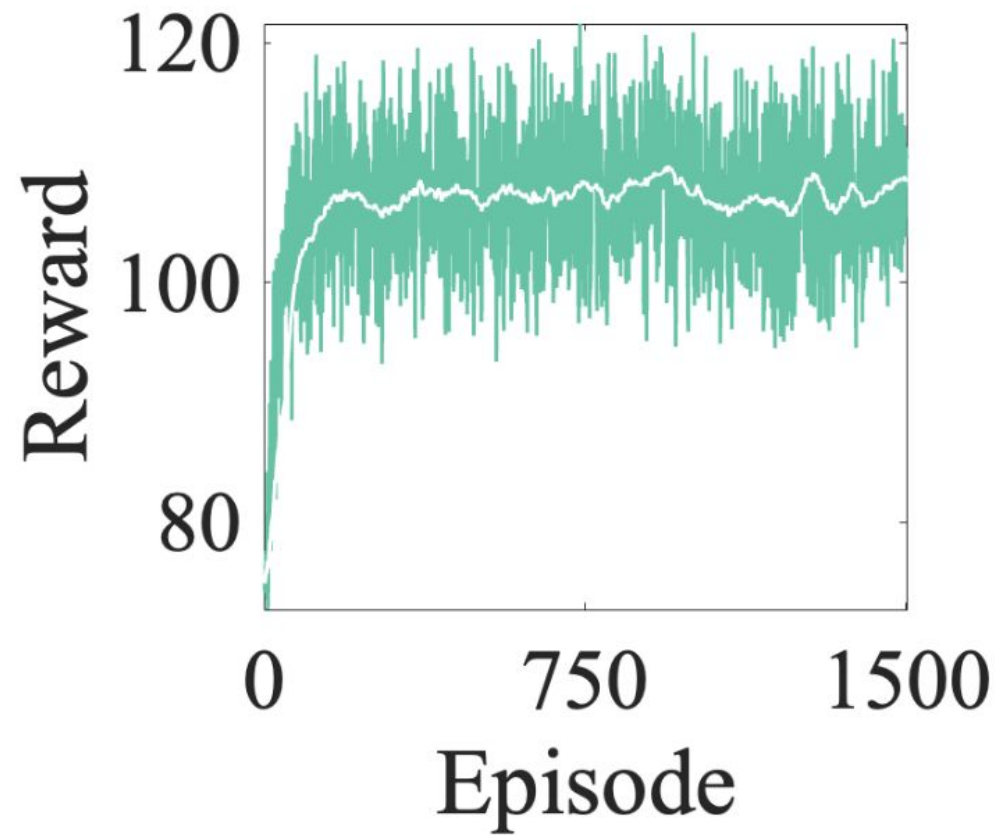
1. Hardware edge testbed

Testbed Setup:

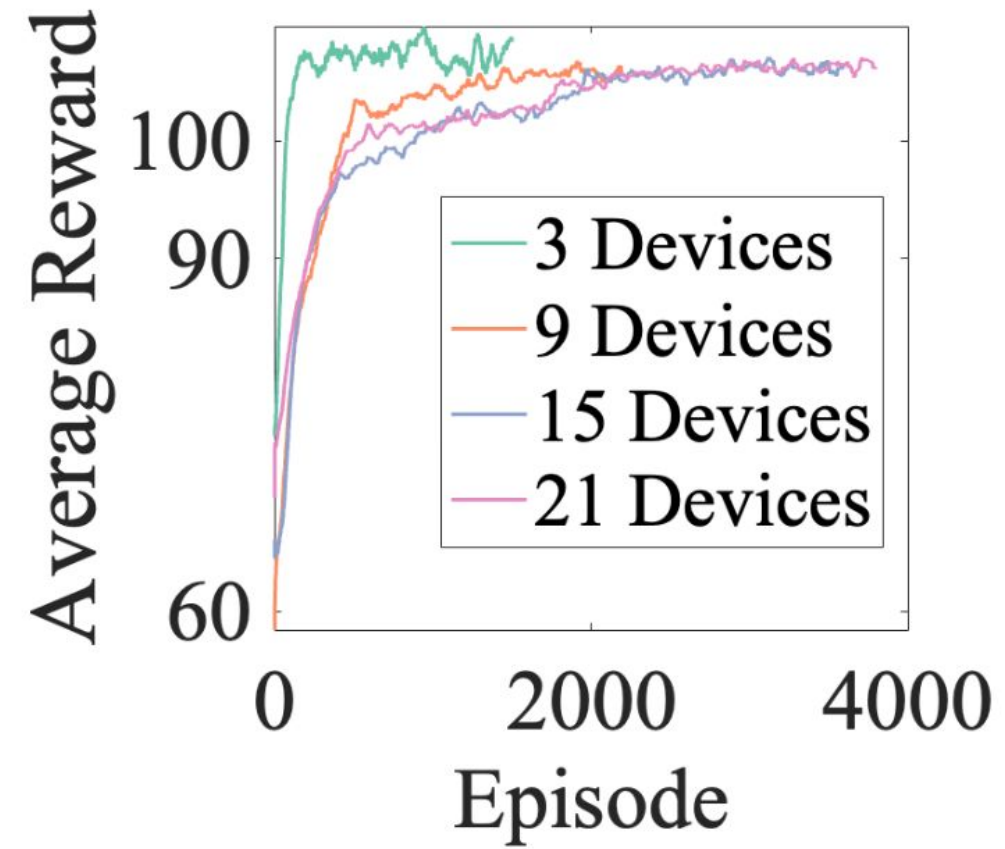
- 3 NVIDIA JETSON TX2
 - 1 Dell PowerEdge Desktop (16 cores and 3.2 GHz CPU)
 - TP-Link router
-
- Controller
 - runs A2C agent
 - deploys DNN Executer
 - VGG, ResNet and DenseNet
 - Virtual Queue
 - Arrival rate (Poisson), Queue size (Exponential)



Learning Stability



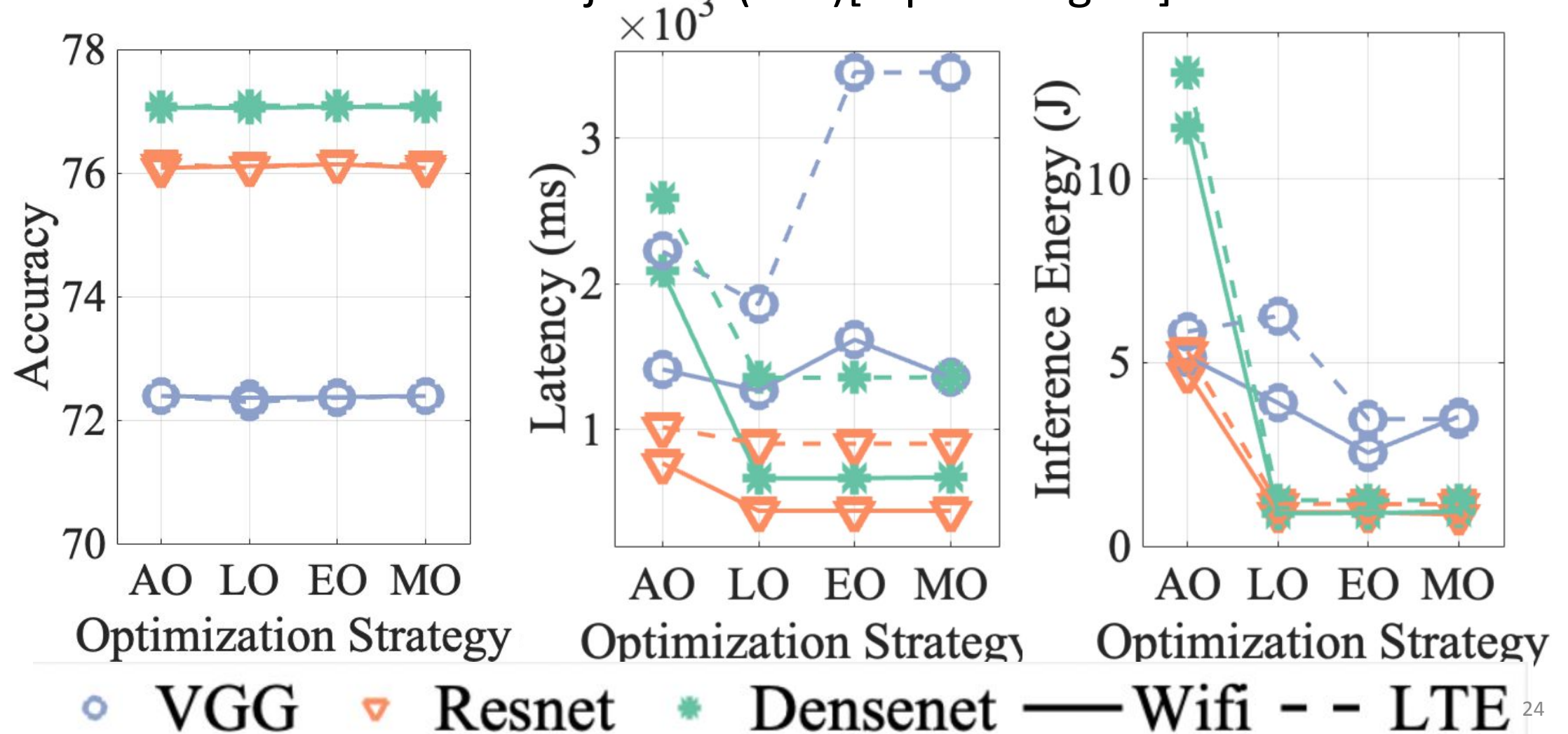
(a)



(b)

Ablation Study

Accuracy-Optimal (AO), Latency-Optimal (LO), Energy Optimal (EO)
Multi-Objective (MO)[equal weights]



Ablation Study

Latency improvement, Energy Saving in comparison to local

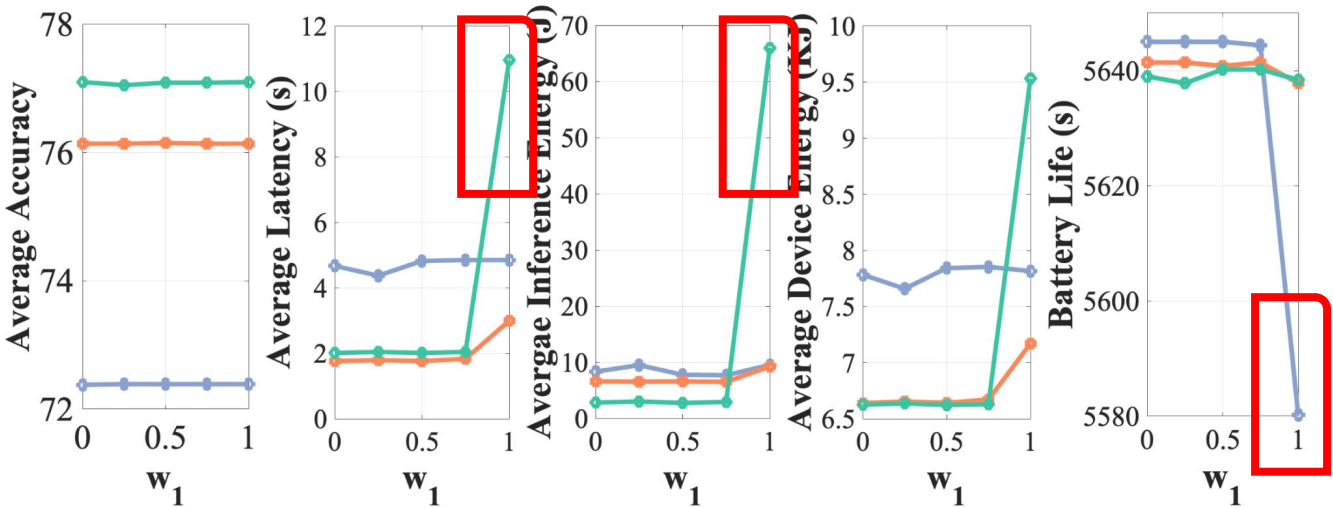
Model	Latency improvement							
	AO		LO		EO		MO	
	LTE	WiFi	LTE	WiFi	LTE	WiFi	LTE	WiFi
Latency	46%	60%	62%	78%	47%	75%	47%	77%
Energy	65%	69%	87%	91%	91%	91%	91%	91%

Energy Saving

Accuracy-Optimal (AO), Latency-Optimal (LO), Energy Optimal (EO)
Multi-Objective (MO)[equal weights]

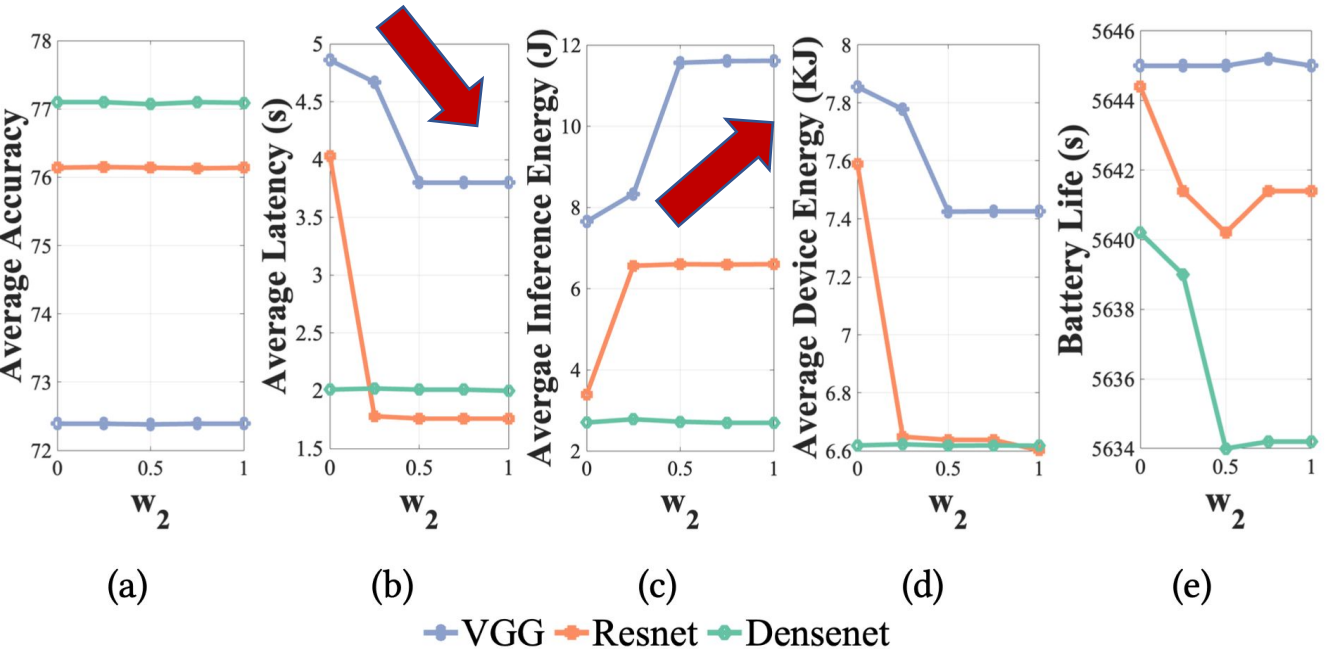
Reward Sensitivity Analysis to weights

Accuracy weight



Accuracy-only drains the battery

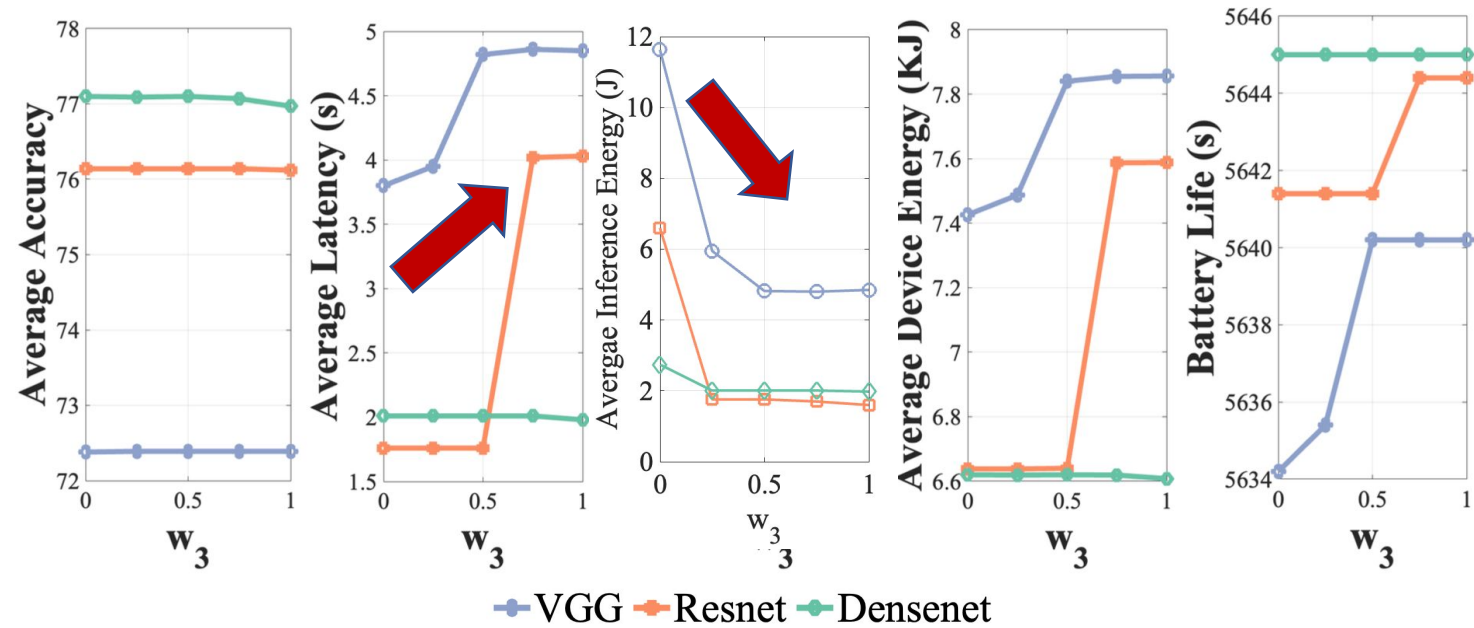
Latency weight



Trade-off between energy and latency

Reward Sensitivity Analysis

Energy weight



Trade-off between energy and latency

Model	Version	Cut Point for			
		$w_2 : 0$	$w_2 : 1$	$w_3 : 0$	$w_3 : 1$
VGG	19	5	10	10	5
ResNet	50	3	13	13	3
DenseNet	161	4	4	4	4

Choice of cut-point for latency-only and energy-only

Conclusion

Infer-EDGE:

- Latency-accuracy-aware energy saving framework
- Resource allocation through execution plan decisions

Customizable framework allows selection between lightweight and heavyweight DNN versions, optimizing latency-accuracy-energy consumption profile.

We verified the benefits of partial task offloading in energy saving:

- Under very **low to barely acceptable data rate** conditions, Infer-EDGE improves the **end-to-end latency by 47%**, and **energy preservation by 91%**.
- Under **moderate to high data rate conditions**, Infer-EDGE achieves up to **77% of latency** reduction and up to **92% improvement** on device energy savings.
- ✓ Infer-Edge framework can strike the balance between accuracy, energy efficiency and inference latency.